

รายงานการวิเคราะห์และตรวจสอบ สถาปัตยกรรม โปรเจกต์ Rnai-CLI

Architectural Inspection & Technical Analysis Report

วันที่จัดทำ: 20 สิงหาคม 2026

เวอร์ชันโปรเจกต์: 0.1.0

สภาพแวดล้อม: macOS / Python

3.9+

Phase 1 Complete

Rnai Agent Architecture

1. ภาพรวมโปรเจกต์ (Project Overview)

Rnai-CLI เป็นระบบ AI Agent CLI และ Web Interface ประสิทธิภาพสูง พัฒนาขึ้นเพื่อเป็นส่วนสำคัญในระบบนิเวศ Rnai.io โดยจัดอยู่ใน Phase 1 ตามแผนการพัฒนา 3 ระยะ (Rnai-CLI → Worker Daemon → Cowork WebApp)

Multi-Model Chat & Compare

สนทนากับ `rnai-llm` บน Modal/Rnai.io API และเปรียบเทียบคำตอบและ Latency ร่วมกับ Groq, Gemini, OpenRouter, Cerebras, Mistral, GitHub Models, Ollama

Hybrid Agent System

ระบบผสมผสานระหว่าง **Planner Brain** (Groq/Gemini สำหรับวางแผน tool-calling) และ **Voice Refinement** (rnai-llm ปรับบุคลิกภาษาไทย)

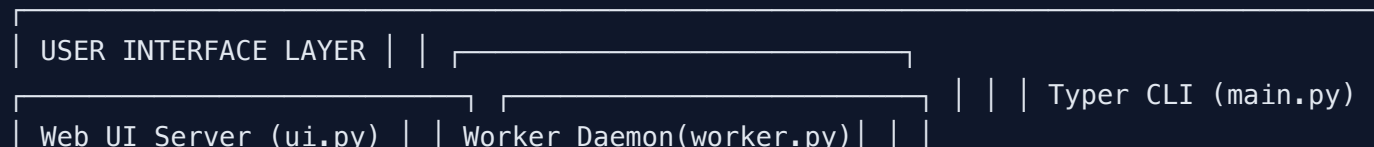
Background Worker & Web UI

ทำงานเบื้องหลังด้วย macOS `Launchd` daemon และ Web UI ในตัวแบบ Zero-Dependency รองรับ PWA บนมือถือและเดสก์ท็อป

2. แผนผังสถาปัตยกรรมระบบ (System Architecture & Operational Flow)

โครงสร้างสถาปัตยกรรมถูกออกแบบในลักษณะ Layered Decoupled Architecture เพื่อให้แต่ละโมดูลทำงานอิสระ มีความปลอดภัยสูง และสลับ Provider ได้อย่างยืดหยุ่น

Rnai-CLI Architectural Operational Flow





3. โครงสร้างไฟล์และหน้าที่ของแต่ละโมดูล (Module Breakdown)

Rnai-CLI/

- └─ pyproject.toml # ตั้งค่า Build System, Dependency (typer, rich, httpx) และ CLI Command Script
- └─ requirements.txt # รายการ Dependency ขั้นต่ำ
- └─ install.sh # Shell script สำหรับติดตั้งโปรเจกต์แบบ Editable mode (pip install -e .)
- └─ README.md # เอกสารแนะนำการติดตั้ง การตั้งค่า Key และตัวอย่างการใช้งาน
- └─ Modelfile.v4.1 / Modelfile.v4_0.1 # Modelfile สำหรับรัน rnai-llm GGUF บน Ollama
- └─ rnai_tools_v1.json # สัญญาเครื่องมือ 6 ชนิดพร้อม Host Guards สำหรับ โปรไฟล์วิจัย
- └─ tests/
 - └─ test_rnai_cli.py # Test suite ครอบคลุม Config, Agent loop, Auth, Tools และ Worker
- └─ rnai_cli/ # โฟลเดอร์ซอร์สโค้ดหลัก
 - └─ __init__.py # Package Initializer (v0.1.0)
 - └─ main.py # CLI Controller (Typer + Rich Interface)
 - └─ config.py # ระบบจัดการ Configuration (~/.rnai/config.json)
 - └─ auth.py # Firebase Auth + Rnai.io Platform API & Billing Integration
 - └─ providers.py # OpenAI-compatible Client, 429 Retry & Tool Call Format Recovery
 - └─ agent.py # Hybrid Agent Loop Controller (Planner + Voice Refiner)
 - └─ tools.py # Agent Tool Implementations & Human-in-the-loop Approval System
 - └─ research_tools.py # Research Profile Tools + Fading Host Guards (Thesis build)
 - └─ templates.py # คลัง Prompt Templates งานสำเร็จรูป (Task/Agent/Chat)
 - └─ history.py # บันทึกและจัดการประวัติการสนทนา JSON (~/.rnai/history/)
 - └─ worker.py # Task Scheduler Queue & macOS launchd daemon manager
 - └─ ui.py # Web App Server (ThreadingHTTPServer + PWA support)

4. การวิเคราะห์ระบบหลักเชิงลึก (Core Component Analysis)

โมดูล	หน้าที่หลัก	การออกแบบสถาปัตยกรรม (Design Patterns)
<code>main.py</code> / <code>config.py</code>	CLI Command Controller & Settings Manager	ใช้ Typer Command Pattern + Rich Console Render, เก็บ config ที่ <code>~/.rnai/config.json</code> โดยบันทึกเฉพาะค่าที่ต่างจาก default เพื่อความปลอดภัย

โมดูล	หน้าที่หลัก	การออกแบบสถาปัตยกรรม (Design Patterns)
<code>auth.py</code>	Authentication & Rnai.io Platform API	ทำ Firebase <code>signInWithPassword</code> นำ Token ไปขอ Mint permanent API Key (<code>rnai_sk_...</code>) และจัดการ <code>platform_chat()</code> กับ Server-side quota fallback
<code>providers.py</code>	Universal LLM Provider Wrapper	OpenAI-Compatible Abstraction Layer, มี Exponential Retry สกัดเวลาจาก <code>Retry-After</code> header และใช้ Regex Tag Parser แกะ Tool Calls ที่หลุดฟอร์แมต
<code>agent.py</code> & <code>tools.py</code>	Hybrid Agent & Tool System	Planner-Voice Architecture: Groq/Gemini ทำ Function Calling multi-steps + สลับ Fallback ชั่วคราวเมื่อ Groq ล้มเหลว, มี Human-in-the-loop approval
<code>research_tools.py</code>	Research Profile & Scaffolding Fading	มี <code>_guard()</code> บังคับ Fading Level (L1, L2, L3) ในระดับโค้ด ป้องกัน AI Hallucination และบันทึก JSONL Log สำหรับใช้เป็นหลักฐานงานวิจัย
<code>worker.py</code>	Background Daemon & Task Queue	รองรับการตั้งเวลา (daily, every N min, once) มีระบบสักรัดเป็น macOS <code>launchd</code> plist และแจ้งเตือนผ่าน OS Notification
<code>ui.py</code>	Single-File Web App & PWA Server	ใช้ <code>ThreadingHTTPServer</code> (Stdlib) ไร้ภาระ Node/NPM มี REST API Endpoint ในตัว พร้อมฝัง Manifest + Service Worker ให้ติดตั้งเป็น PWA ได้

💡 ไฮไลต์สถาปัตยกรรมสำคัญ: Hybrid Planner + Voice Paradigm

ระบบแยกส่วนการ "วางแผนและคิด" (Planner) ออกจาก "การเรียบเรียงน้ำเสียง" (Voice) อย่างชัดเจน:

- **Planner:** ใช้โมเดลที่เก่งด้าน Tool-Calling และ Logic (เช่น Llama-3.3-70B บน Groq หรือ Gemini-2.5-Flash) เพื่อทำงานให้เสร็จเร็วที่สุด
- **Voice:** ใช้โมเดล `rnai-llm` เรียบเรียงผลลัพธ์ดั้งเดิมจาก Planner ให้กลายเป็นภาษาไทยที่เป็นธรรมชาติ สไตล์ Rnai โดยไม่ดัดแปลงตัวเลข ข้อเท็จจริง หรือลิงก์

5. จุดเด่นและข้อเสนอแนะในการพัฒนา (Strengths & Recommendations)

✨ จุดเด่นทางสถาปัตยกรรม (Key Strengths)

- **Zero-Heavy Dependencies:** ใช้เพียง `typer`, `rich`, `httpx` ติดตั้งง่าย รวดเร็ว และไม่มีปัญหาความขัดแย้งของไลบรารี
- **High Fault Tolerance:** มี Retry ลูปสำหรับ HTTP 429, มี Fallback Planner ข้ามค่าย (Groq → Gemini) และมี Regex สกัด Tool Call Recovery
- **Layered Security:** กิจกรรมที่แก้ไขระบบ ต้องได้รับการยืนยันจากผู้ใช้ก่อนเสมอ และโหมด Worker เบื้องหลังจะปิด Shell Command โดยอัตโนมัติ
- **Portable Web UI / PWA:** รัน WebApp ได้ทันทีโดยไม่ต้องลง Node.js หรือสร้าง Build Pipeline

🚀 ข้อเสนอแนะพัฒนาต่อ (Next Steps)

- **UI Code Modularization:** แยก HTML/CSS/JS จาก `ui.py` ออกเป็นไฟล์ static template เพื่อการบำรุงรักษาที่ง่ายขึ้น
- **Server-Sent Events (SSE) Streaming:** เพิ่มระบบ Streaming Response ใน Web UI สำหรับขั้นตอนการคิดของ Agent ให้เห็นภาพแบบ Real-time
- **Linux / Windows Daemon Support:** ขยายระบบตั้งเวลาจาก macOS `launchd` ให้รองรับ `systemd` service สำหรับนำไปรันบน Linux Cloud Server